THADOMAL SHAHANI
TSEC
ENGINEERING COLLEGE

Assignment 8

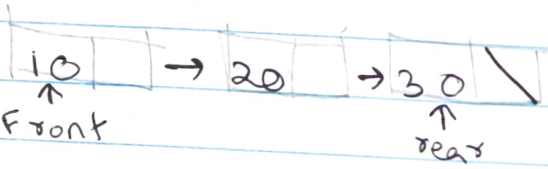
Aim - Write a menu driven program to implement doubly ended queue using linked list.

Theory

In a doubly ended queue, insertion and deletion can occur at both the front and rear ends. This is implemented using linked lists

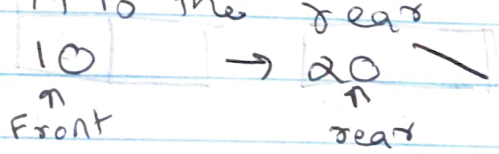
Input restricted Queue

It allows insertion on one end while deletion can occur from both front and rear



Deletion from front - It moves front to the next node and frees the previous front

Deletion from rear - It sets the next of the second last ~~entry~~ node to null indicating it is the rear



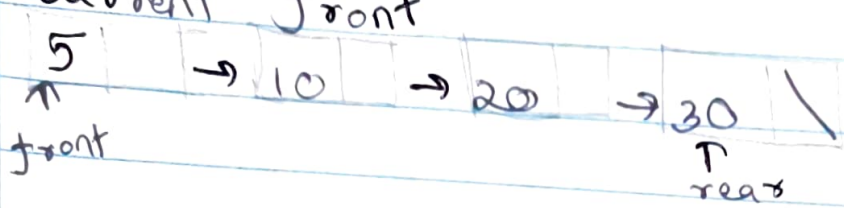
Output Restricted Queue

Insertion is allowed ~~only~~ at ~~one~~ both ends while

deletion is allowed at only one end (usually rear)

Insert at front

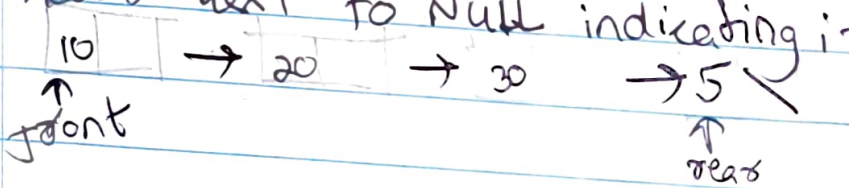
Creates a new node and sets its next to the previous front, makes the new node the current front



(Insertion of 5)

Insertion at end.

Creates a new node and sets the next of the rear to that node and makes the nodes next to NULL indicating it is the ending.



(Insertion of 5)

Time Complexity

O(1) - Insertion at front, rear and deletion from front

O(n) - Deletion from rear (In singly linked list), Display (Traversal is required)

Conclusion

The program performs insertion and deletion at both the front and rear of the singly linked list as well as displays. It demonstrates the working of input and output restricted queues.

```
#include<stdio.h>
#include<stdlib.h>
struct node{
   int data;
   struct node* next;
};
void enqFront(struct node** front, struct node** rear, int value);
void enqRear(struct node** front, struct node** rear, int value);
void deqFront(struct node** front, struct node** rear);
void deqRear(struct node** front, struct node** rear);
void display(struct node* front);
int main()
{
   int n, value;
   struct node* front = NULL;
   struct node* rear = NULL;
   while(1)
   {
      printf("\n1. Enqueue at front");
      printf("\n2. Enqueue at rear");
      printf("\n3. Dequeue at front");
      printf("\n4. Dequeue at rear");
      printf("\n5. Display");
      printf("\n6. Exit\n");

      printf("Enter your choice: ");
      scanf("%d",&n);

      switch(n)
      {
```

```
        case 1:
          printf("Enter value: ");
          scanf("%d",&value);
          enqFront(&front,&rear,value);
          break;
        case 2:
          printf("Enter value: ");
          scanf("%d",&value);
          enqRear(&front,&rear,value);
          break;
        case 3:
          deqFront(&front,&rear);
          break;
        case 4:
          deqRear(&front,&rear);
          break;
        case 5:
          display(front);
          break;
        case 6:
          exit(0);
        default:
          printf("Invalid choice\n");
      }
  }
}
void enqFront(struct node** front, struct node** rear, int value)
{
  struct node* newnode = (struct node*)malloc(sizeof(struct node));
  newnode->data = value;
  newnode->next = *front;
```

```
    if(*front == NULL)
    {
        *front = *rear = newnode;
    }
    else
    {
        newnode->next = *front;
        *front = newnode;
    }
}

void enqRear(struct node** front, struct node** rear, int value)
{
    struct node* newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = value;
    newnode->next = NULL;
    if(*rear == NULL)
    {
        *front = *rear = newnode;
    }
    else
    {
        (*rear)->next = newnode;
        *rear = newnode;
    }
}

void deqFront(struct node** front, struct node** rear)
{
    if(*front == NULL)
    {
```

```
        printf("Deque is empty\n");
        return;
    }
    struct node* temp = *front;
    printf("Deleted element: %d\n", temp->data);
    *front = (*front)->next;
    if(*front == NULL)
        *rear = NULL;
    free(temp);
}
void deqRear(struct node** front, struct node** rear)
{
    if(*rear == NULL)
    {
        printf("Deque is empty\n");
        return;
    }
   if(*front == *rear)
    {
        printf("Deleted element: %d\n", (*rear)->data);
        free(*rear);
        *front = *rear = NULL;
        return;
    }
    struct node* temp = *front;
    while(temp->next != *rear)
        temp = temp->next;
    printf("Deleted element: %d\n", (*rear)->data);
    free(*rear);
    temp->next = NULL;
    *rear = temp;
```

```
}
void display(struct node* front)
{
   if(front == NULL)
   {
      printf("Queue is empty\n");
      return;
   }
   struct node* temp = front;
   while(temp != NULL)
   {
      printf("%d ",temp->data);
      temp = temp->next;
   }
   printf("\n");
}
```

```
1. Enqueue at front
2. Enqueue at rear
3. Dequeue at front
4. Dequeue at rear
5. Display
6. Exit
Enter your choice: 1
Enter value: 10

1. Enqueue at front
2. Enqueue at rear
3. Dequeue at front
4. Dequeue at rear
5. Display
6. Exit
Enter your choice: 1
Enter value: 20

1. Enqueue at front
2. Enqueue at rear
3. Dequeue at front
4. Dequeue at rear
5. Display
6. Exit
Enter your choice: 2
Enter value: 30

1. Enqueue at front
2. Enqueue at rear
3. Dequeue at front
4. Dequeue at rear
5. Display
6. Exit
Enter your choice: 5
20 10 30

1. Enqueue at front
2. Enqueue at rear
3. Dequeue at front
4. Dequeue at rear
5. Display
6. Exit
Enter your choice: 3
Deleted element: 20
```

```
1. Enqueue at front
2. Enqueue at rear
3. Dequeue at front
4. Dequeue at rear
5. Display
6. Exit
Enter your choice: 5
10 30

1. Enqueue at front
2. Enqueue at rear
3. Dequeue at front
4. Dequeue at rear
5. Display
6. Exit
Enter your choice: 4
Deleted element: 30

1. Enqueue at front
2. Enqueue at rear
3. Dequeue at front
4. Dequeue at rear
5. Display
6. Exit
Enter your choice: 4
Deleted element: 10

1. Enqueue at front
2. Enqueue at rear
3. Dequeue at front
4. Dequeue at rear
5. Display
6. Exit
Enter your choice: 5
Queue is empty

1. Enqueue at front
2. Enqueue at rear
3. Dequeue at front
4. Dequeue at rear
5. Display
6. Exit
Enter your choice: 6
```